

NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis

20 June 2026 08:04 PM

➤ What we are working on?

- We want to generate a novel view of objects with respect to unseen angle and positions
- This can be done by Learning "**continuous volumetric scene function**" that we can query using 5 co-ordinates (3 position vector + 2 viewing angle) and it will render the scene for that set of "co-ordinates + Viewing Angle"
- This is the entire idea behind the NeRF Paper
- **The output:** It will not be the image itself, but we need to use some volume rendering techniques to project the (volume density + view-dependent emitted radiance)

The Entire Problem: If I shoot this ray into the scene, what colour should this pixel become?

The Pipeline (in detail):

- Step 1: One Image

- **Image of resolution (800 x 800): Total pixels we have = 640,000** (each pixel corresponds to one ray, so we have total of 640,000 such Rays)
- In order to render the image, we ask "**If I shoot this ray into the scene, what colour should this pixel become?**"
 - Pixel (0,0) → Ray 1
 - Pixel (0,1) → Ray 2
 - Pixel (799,799) → Ray 640,000

- Step 2: One Ray

- Along this Ray, we ask at several locations:
 - Is there any matter here? (density σ)
 - If light comes from here toward the camera, what colour would it be? (radiance c)
- Consider one pixel, Generate the corresponding ray, that created this pixel
- We Sample ' $N = 64$ ' points on this ray (for every sampled points, the viewing direction is same)
 - Sampled points are:
 - (Camera) ● — x_1 — x_2 — x_3 — ... — x_{64} —► [R, G, B]
 - If we choose $N = 64$, then we have 64 sampled 3D points and we can Query out MLP on all these 64 Points and get the "**Density + RGB value**"

- Step 3: Query the MLP

- Each sampled point becomes one input to the MLP network
- NOTE: **Because they're all on the same ray, we can say that**
 - Position changes
 - Direction stays the same

- Step 4: Combine them

Point	Density	Color
x_1	σ_1	RGB ₁
x_2	σ_2	RGB ₂
....		
x_{64}	σ_{64}	RGB ₆₄

- Now **Volume rendering** computes "**Final Pixel RGB**" using all 64 predictions

- Summary

- We know the ray from the camera geometry. Since a ray contains infinitely many 3D points, we sample a finite number of points along it. At each sampled point, the **NeRF** network predicts the local density and colour. These predictions are then integrated (approximately, using sampled points) to estimate the final colour of the pixel.

➤ Mathematically,

Dataset

$$D = \{(\mathcal{I}_i, P_i)\}_{i=1}^M$$

$\mathcal{I}_i$: i th image

P_i : camera pose of i th image (rotation + translation)

Ray-Generation :-

for each pixel, we generate 'Ray' using camera parameters.

$$\gamma(t) = o + td \rightarrow \text{'Ray'}$$

Camera origin distance from camera ray direction

Sample points along the Ray

Sampled points are:

$$\{t_1, t_2, t_3 \dots t_N\}_N \text{ sampled points.}$$

their positions are,

$$X_i = o + t_i d \rightarrow \text{Give MLP on these points.}$$

Query MLP

F_θ : MLP

$$(\sigma_i, c_i) = F_\theta(X_i, d)$$

Volume
Density

Radiance
field (RGB).

Use Volume rendering techniques
→ to model the pixel value from
this MLP output.

now we have,

$$\{(\sigma_1, c_1), (\sigma_2, c_2), (\sigma_3, c_3) \dots (\sigma_N, c_N)\}$$

for each 'N' points $\{X_1, X_2, \dots, X_N\}$

Compute Transmittance,

how much light reaches sample 'i',
light must pass from all the previous sample.

Define,

$$T_i = e^{-\sum_{j=1}^{i-1} \sigma_j S_j}$$

where,

$$S_i = T_{i+1} - T_i$$

if $T_i = 0$; light gets blocked with all
the previous samples

if $T_i = 1$; No obstruction

Compute Sample weight,

Compute Sample weight,

$$w_i = \underbrace{\tau_i}_{\text{light reaching the sample}} \underbrace{(1 - e^{-\sigma_i s_i})}_{\text{probability of 'Ray' terminating into the matter.}}$$

Render Pixel Colour:-

$$\hat{C}(x) = \sum_{i=1}^N w_i C_i \quad \begin{array}{l} \text{from the MLP} \\ \text{weight of that sample} \end{array}$$

Compute Loss

$$\mathcal{L} = \|\hat{C}(x) - C(x)\|_2^2$$

Backpropagation,

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla_{\theta} \mathcal{L}$$

○ What do we mean by "Volume density and view-dependent emitted radiance"?

▪ Volume Density:

- How much matter exists at that point
- **It Learns:** How likely is light to stop here?
- Mathematically,
 - ◆ $\sigma = 0$ (No Object, light will not stop at that point and will pass)
 - ◆ **Large (σ)** => Object surface, light will get absorb and will **reflect back to the camera** (positioned at viewing angle + 3d position vector)
 - ◆ Now the next obvious question is, "**What colour of light leaves this point**"

toward my eye?" ----> That is "Radiance"

- **Radiance:**

- Radiance simply means, Colour leaving from any specific point (R, G, B)

- **Why is it "View-dependent"?**

- Most objects do **not** look identical from every angle.

➤ **The end-to-end pipeline:**

- 3D (Position vector) + 2D (Camera Angle) ----> **NeRF** ----> volume density + view-dependent emitted radiance ----> Volume rendering techniques (mathematically differentiable) ----> Rendered Image

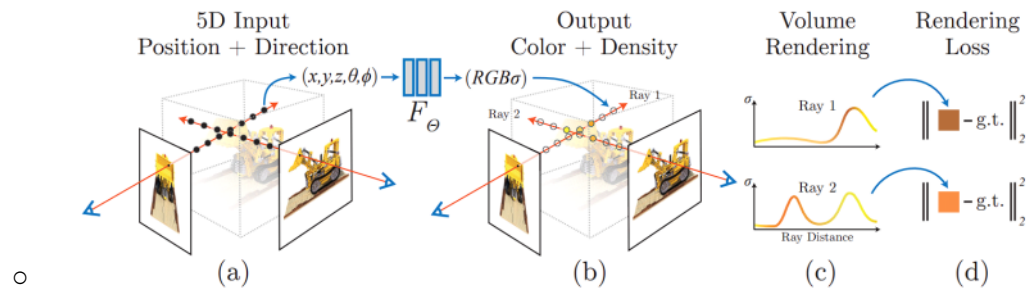


Fig. 2: An overview of our neural radiance field scene representation and differentiable rendering procedure. We synthesize images by sampling 5D coordinates (location and viewing direction) along camera rays (a), feeding those locations into an MLP to produce a color and volume density (b), and using volume rendering techniques to composite these values into an image (c). This rendering function is differentiable, so we can optimize our scene representation by minimizing the residual between synthesized and ground truth observed images (d).